

TUGAS KECIL 2
IF2211 STRATEGI ALGORITMA
Ice Sliding Puzzle Solver



Disusun oleh :
Kholida Rezki Khoiriah (13222071)

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

Daftar Isi

BAB I Algoritma Divide and Conquer dalam Voxelization menggunakan Octree	3
BAB II Source Program	5
BAB III: Pengujian dan Hasil Pengujian	15
Lampiran	21

BAB I

Algoritma *Pathfinding* dalam *Ice Sliding Puzzle Solver*

Ice Sliding Puzzle adalah gim logika bersimulasi es licin yang menuntut pemain menggerakkan pin biru dari titik awal ke titik keluar melalui gerakan horizontal atau vertikal. Karakter yang digerakkan akan terus meluncur tanpa henti akibat permukaan yang licin, dan hanya akan menetap di posisi baru setelah menabrak dinding atau rintangan putih yang menghalanginya.

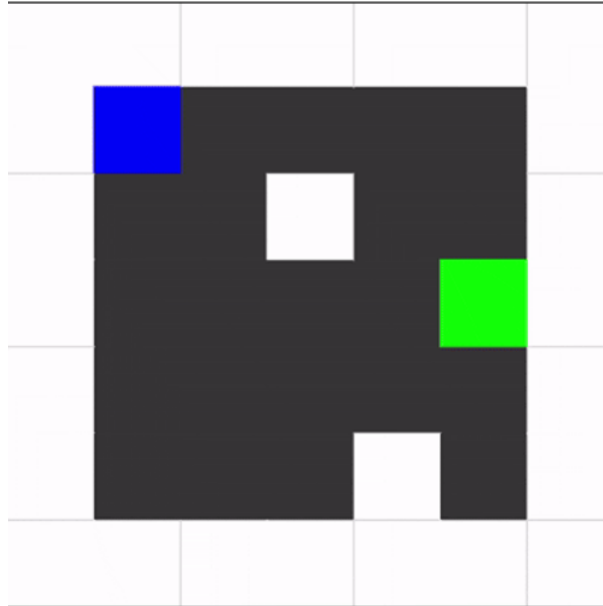


Figure 1 Visualisasi Permainan *Ice Sliding Puzzle Solver*

1. Representasi Persoalan

Sebuah **state** dalam *solver* ini direpresentasikan sebagai *tuple* tiga nilai: (**row**, **col**, **next_num**). *row* dan *col* menyatakan posisi baris dan kolom aktor saat ini di papan, sementara *next_num* menyatakan angka berikutnya yang wajib diinjak.

Posisi saja tidak cukup karena papan memiliki *constraint* urutan angka. Dua situasi di mana aktor berada di koordinat yang sama bisa benar-benar berbeda. Misalnya, berada di (3, 2) sebelum mengambil angka 0 adalah *state* yang berbeda dengan berada di (3, 2) setelah mengambil angka 0. Jika hanya koordinat yang disimpan, algoritma tidak bisa membedakan keduanya dan akan salah menandai *state* sebagai "sudah dikunjungi" padahal *progress*-nya berbeda.

Selanjutnya untuk mekanisme *sliding*. Ketika aktor bergerak ke suatu arah, ia meluncur ubin demi ubin sampai terhenti. Aktor berhenti jika langkah berikutnya adalah dinding (X), ia berhenti tepat satu ubin sebelum dinding. Aktor mengalami game over (gerakan dianggap tidak valid) jika:

- 1) Meluncur keluar batas papan tanpa ada dinding yang menghentikan,
- 2) Melewati ubin lava (L), atau
- 3) Melewati ubin angka yang nilainya lebih besar dari *next_num*, melanggar urutan yang diwajibkan.

Cost satu gerakan dihitung sebagai penjumlahan *cost* seluruh ubin yang dilewati selama meluncur, tidak termasuk ubin posisi awal, tetapi termasuk ubin posisi akhir (tempat berhenti). Jika aktor meluncur melewati tiga ubin dengan *cost* 2, 5, dan 4, maka *cost* gerakan itu adalah 11. Total *cost* solusi adalah jumlah *cost* seluruh gerakan.

2. Uniform Cost Search (UCS)

UCS mencari jalur dengan total *cost* paling murah tanpa menggunakan heuristik. Algoritma ini menjamin solusi optimal selama semua *cost* bernilai non-negatif. UCS menggunakan *priority queue* (*min-heap*) yang diurutkan berdasarkan $g(n)$, yaitu total *cost* riil yang sudah dikeluarkan dari posisi awal hingga *state* n . *State* dengan $g(n)$ terkecil selalu diproses lebih dulu.

Langkah-langkahnya:

- 1) Masukkan *state* awal ke *priority queue* dengan $g = 0$.
- 2) *Pop state* dengan g terkecil dari *heap*.
- 3) Cek apakah *state* ini adalah *goal*. Jika ya, *return* solusi.
- 4) Jika *state* ini sudah pernah dikunjungi dengan *cost* yang sama atau lebih murah, *skip*.
- 5) Tandai *state* sebagai *visited*.
- 6) Ekspan ke 4 arah menggunakan fungsi *slide()*. Untuk setiap arah yang valid, hitung $g_{baru} = g_{lama} + move_cost$, lalu *push* ke *heap*. Ulangi dari langkah 2.

3. Greedy Best-First Search (GBFS)

GBFS juga menggunakan *priority queue*, tetapi urutan prioritasnya bukan $g(n)$ melainkan $h(n)$, yaitu estimasi jarak dari *state* saat ini ke tujuan. Algoritma ini serakah/*greedy*, maksudnya selalu memilih *state* yang *terlihat* paling dekat ke *goal*, tanpa memedulikan berapa *cost* yang sudah dikeluarkan untuk sampai ke sana.

Akibatnya, GBFS biasanya lebih cepat menemukan solusi (iterasi lebih sedikit), tetapi tidak menjamin solusi yang ditemukan adalah yang paling murah. Ia bisa "tertipu" oleh estimasi heuristik dan melewati jalur yang lebih murah.

Perbedaan utama dari UCS adalah *visited* hanya dicek satu kali (tanpa membandingkan *cost*), karena GBFS tidak melacak $g(n)$ per *state*. Akibatnya jika sebuah *state* sudah pernah dikunjungi, ia langsung di-*skip* tanpa memeriksa apakah ada jalur lebih murah ke sana.

4. A* (A-Star)

A* menggabungkan kelebihan UCS dan GBFS. *Priority queue* diurutkan berdasarkan $f(n) = g(n) + h(n)$, yaitu gabungan antara *cost* riil yang sudah dikeluarkan (g) dan estimasi sisa jarak ke *goal* (h). Dengan mempertimbangkan $g(n)$, A* tidak mudah "tersesat" mengejar target yang tampak dekat padahal butuh *cost* besar. Dengan mempertimbangkan $h(n)$, A* lebih terarah dibanding UCS yang mengeksplorasi ke semua arah secara merata. Hasilnya: A* biasanya lebih efisien dari UCS (lebih sedikit iterasi) dan lebih optimal dari GBFS (menjamin solusi termurah), selama heuristik yang dipakai bersifat *admissible*.

5. Fungsi Heuristik

Fungsi heuristik yang digunakan adalah *Manhattan Distance* dengan target yang bersifat dinamis. Target ditentukan berdasarkan nilai *next_num*: jika masih ada angka yang belum diambil ($next_num < total_nums$), maka target adalah posisi angka *next_num* di papan. Jika semua angka sudah diambil ($next_num == total_nums$), maka target adalah posisi *goal* 0.

Rumus: $h(n) = |row_sekarang - row_target| + |col_sekarang - col_target|$. Heuristik ini hanya melihat satu langkah ke depan (angka berikutnya atau *goal*), bukan keseluruhan urutan yang harus diselesaikan.

BAB II

Analisis Algoritma

1. Definisi $f(n)$ dan $g(n)$

$g(n)$ adalah total *cost* riil yang sudah dikeluarkan dari *state* awal hingga *state* n . Dalam *solver* ini, $g(n)$ dihitung sebagai penjumlahan *cost* semua *tile* yang dilalui selama seluruh gerakan *sliding* dari posisi *start* sampai posisi n . Setiap *tile* yang dilewati (termasuk *tile* berhenti, tidak termasuk *tile* awal gerakan) menyumbangkan *cost* sesuai nilai di *cost grid*.

$f(n)$ adalah estimasi total *cost* lintasan terbaik yang melewati *state* n , didefinisikan sebagai $f(n) = g(n) + h(n)$. Nilai $g(n)$ adalah yang sudah pasti (*cost* yang sudah terjadi), sedangkan $h(n)$ adalah estimasi sisa *cost* dari n ke goal. A* menggunakan $f(n)$ sebagai kunci pengurutan priority queue, sehingga ia selalu memprioritaskan *state* yang diprediksi memiliki total lintasan paling murah.

2. Admissibility Heuristik

Heuristik *Manhattan Distance* yang dipakai bersifat *admissible* untuk persoalan ini. Sebuah heuristik *admissible* berarti ia tidak pernah mengestimasi *cost* lebih besar dari *cost* riil minimum. Artinya: $h(n) \leq h^*(n)$ untuk semua *state* n , di mana $h^*(n)$ adalah *cost* riil minimum dari n ke *goal*.

Dalam *Ice Sliding Puzzle*, aktor meluncur melewati *tile-tile* dengan *cost* tertentu. Dalam satu gerakan *sliding*, aktor melewati satu atau lebih *tile*. *Manhattan Distance* menghitung jumlah langkah minimum tanpa memperhitungkan *cost* tiap *tile*. Karena *cost* setiap *tile* minimal 1 (dan dalam praktiknya sering lebih besar), *Manhattan Distance* selalu merupakan batas bawah dari *cost* riil, artinya jarak tidak pernah melebihi-lebihkan atau bisa juga disebut *admissible*. Oleh karena itu, A* dengan heuristik ini menjamin solusi optimal: jika solusi ditemukan, ia adalah jalur dengan total *cost* paling murah.

3. Perbandingan Algoritma (Teoritis)

UCS dan A* keduanya *complete*. Artinya jika solusi ada dan *graph* terhingga, keduanya pasti menemukannya. GBFS juga *complete* pada *graph* terhingga, tetapi bisa terjebak di *cycle* pada *graph* tak terhingga jika tidak dilengkapi deteksi *visited* yang tepat.

UCS menjamin solusi optimal (*cost* terkecil) karena ia mengeksplorasi berdasarkan $g(n)$. A* juga menjamin optimal selama heuristik *admissible*. GBFS tidak menjamin optimal, ia bisa menemukan solusi yang lebih cepat secara iterasi, tetapi belum tentu yang termurah.

UCS pada kasus terburuk mengeksplorasi semua *state* dengan $cost \leq cost$ solusi optimal, sehingga bisa sangat lambat di papan besar. GBFS umumnya paling cepat mencapai *goal* karena langsung mengejar estimasi terdekat, tetapi bisa eksplorasi banyak *state* yang tidak relevan jika heuristik kurang akurat. A* biasanya mengeksplorasi jauh lebih sedikit *state* dibanding UCS berkat panduan heuristik, sambil tetap menjamin optimalitas.

Kesimpulannya, UCS unggul saat *cost* antar *tile* sangat bervariasi dan optimalitas mutlak dibutuhkan. GBFS unggul saat kecepatan lebih penting dari optimalitas, misalnya untuk *puzzle* besar yang membutuhkan respons cepat. A* adalah pilihan terbaik secara umum: efisien dan tetap optimal.

BAB III

Source Program

Program ditulis menggunakan bahasa pemrograman Python. Daftar Module/Library :

- **typing** (NamedTuple) digunakan untuk mendefinisikan *State* sebagai struktur data *immutable* yang *hashable*, sehingga bisa dipakai sebagai *key dictionary* dan elemen set.
- **heapq** merupakan modul bawaan Python untuk implementasi *min-heap (priority queue)*. Digunakan di UCS, GBFS, dan A* untuk selalu memproses *state* dengan prioritas terkecil terlebih dahulu.
- **time** digunakan untuk mengukur waktu eksekusi algoritma dalam milidetik, tidak termasuk waktu baca *file* dan tampilkan *output*

Source code :

```
from typing import NamedTuple
import heapq
import time

def read_input(filepath):
    f = open(filepath)
    lines = f.readlines()
    f.close()

    # Baris pertama jadikan integer
    baris_pertama = lines[0].split()
    N = int(baris_pertama[0])
    M = int(baris_pertama[1])

    # Baca N baris berikutnya (grid karakter)
    grid = []
    for i in range(1, N + 1):
        baris = lines[i].strip()
        row = list(baris)
        grid.append(row)

    # Baca N baris berikutnya (grid cost)
    cost_grid = []
    for i in range(N + 1, 2 * N + 1):
        baris = lines[i].strip()
        angka_angka = baris.split()
        row = [int(x) for x in angka_angka]
        cost_grid.append(row)

    # Cari posisi Z, O, dan angka-angka dengan loop seluruh grid
    start = None # akan diisi (baris, kolom) posisi Z
    goal = None # akan diisi (baris, kolom) posisi O
```

```

numbers = {}    # akan diisi {angka: (baris, kolom)}

for r in range(N):
    for c in range(M):
        karakter = grid[r][c]
        if karakter == 'Z':
            start = (r, c)
        elif karakter == 'O':
            goal = (r, c)
        elif karakter.isdigit():
            numbers[int(karakter)] = (r, c)
total_nums = len(numbers) # banyak angka di peta
return N, M, grid, cost_grid, start, goal, numbers, total_nums

class State(NamedTuple):
    row: int        # posisi baris aktor sekarang
    col: int        # posisi kolom aktor sekarang
    next_num: int   # angka berikutnya yang harus diinjak

# Arah gerak sebagai (dr, dc)
atas    = (-1, 0)
bawah   = ( 1, 0)
kiri     = ( 0, -1)
kanan    = ( 0, 1)

def slide(grid, cost_grid, state, direction):
    N = len(grid)
    M = len(grid[0])
    dr, dc = direction
    r      = state.row
    c      = state.col
    next_num = state.next_num
    total_cost = 0
    while True:
        # Hitung posisi satu langkah ke depan
        r_baru = r + dr
        c_baru = c + dc

        # Game over jika keluar batas papan
        if r_baru < 0 or r_baru >= N or c_baru < 0 or c_baru >= M:
            return None

        tile = grid[r_baru][c_baru]
        # Jika dinding (X), berhenti di posisi sebelum ini
        if tile == 'X':

```

```

        if r == state.row and c == state.col:
            return None
        state_baru = State(row=r, col=c, next_num=next_num)
        return state_baru, total_cost
    # Jika lava (L), game over
    if tile == 'L':
        return None
    # Kalau angka
    if tile.isdigit():
        angka = int(tile)
        if angka == next_num:
            next_num += 1      # angka berhasil diambil, lanjut slide
        elif angka > next_num:
            return None        # urutan dilanggar, game over
        # kalau angka < next_num → sudah pernah diambil, anggap tile biasa,
        lanjut
        # Tambahkan cost tile yang baru diinjak
        total_cost += cost_grid[r_baru][c_baru]
        # Pindah ke posisi baru, lanjut loop
        r = r_baru
        c = c_baru

def ucs(grid, cost_grid, start, goal, total_nums):
    state_awal = State(row=start[0], col=start[1], next_num=0)
    # Priority queue isinya tuple (g_cost, state, moves_string)
    # heapq selalu pop yang g_cost-nya paling kecil duluan
    pq = []
    heapq.heappush(pq, (0, state_awal, ""))

    # visited: key = state, value = g_cost terkecil yang pernah nyampe ke state itu
    visited = {}
    iterasi = 0
    while pq:
        g_cost, state, moves = heapq.heappop(pq)
        iterasi += 1

        # Cek goal: posisi di 0 dan semua angka sudah diambil
        if state.row == goal[0] and state.col == goal[1] and state.next_num ==
total_nums:
            return moves, g_cost, iterasi

        # Kalau state ini sudah pernah dikunjungi dengan cost lebih murah, skip
        if state in visited and visited[state] <= g_cost:
            continue

```



```

        # Tandai state ini sudah dikunjungi dengan cost ini
        visited[state] = g_cost

        # Expand ke 4 arah
        for nama_arah, arah in [("U", atas), ("D", bawah), ("L", kiri), ("R",
kanan)]:
            hasil = slide(grid, cost_grid, state, arah)
            if hasil is None:
                continue # arah ini invalid, skip
            state_baru, move_cost = hasil
            g_cost_baru = g_cost + move_cost
            # Kalau state baru belum pernah dikunjungi, atau ketemu jalur lebih
murah
            if state_baru not in visited or visited[state_baru] > g_cost_baru:
                heapq.heappush(pq, (g_cost_baru, state_baru, moves + nama_arah))

        # Heap kosong, tidak ada solusi
        return None, 0, iterasi

def heuristic(state, goal, numbers, total_nums):
    # Tentukan target dulu: posisi angka berikutnya, atau goal kalau semua sudah
diambil
    if state.next_num < total_nums:
        target = numbers[state.next_num]
    else:
        target = goal
    # Manhattan distance dari posisi sekarang ke target
    return abs(state.row - target[0]) + abs(state.col - target[1])

def gbfs(grid, cost_grid, start, goal, numbers, total_nums):
    state_awal = State(row=start[0], col=start[1], next_num=0)
    # Priority queue isinya tuple (h_cost, state, moves_string)
    # GBFS hanya pakai h(n) – estimasi jarak ke goal, tanpa memperhitungkan cost
sejauh ini
    pq = []
    h_awal = heuristic(state_awal, goal, numbers, total_nums)
    heapq.heappush(pq, (h_awal, state_awal, ""))
    visited = {}
    iterasi = 0
    while pq:
        h_cost, state, moves = heapq.heappop(pq)
        iterasi += 1
        # Cek goal
        if state.row == goal[0] and state.col == goal[1] and state.next_num ==
total_nums:

```

```

        # GBFS tidak tracking g_cost, jadi hitung ulang total cost dari moves
        total_cost = hitung_total_cost(grid, cost_grid, start, moves)
        return moves, total_cost, iterasi
    if state in visited:
        continue

    visited[state] = True

    for nama_arah, arah in [("U", atas), ("D", bawah), ("L", kiri), ("R",
kanan)]:
        hasil = slide(grid, cost_grid, state, arah)
        if hasil is None:
            continue

        state_baru, move_cost = hasil

        if state_baru not in visited:
            h_baru = heuristic(state_baru, goal, numbers, total_nums)
            heapq.heappush(pq, (h_baru, state_baru, moves + nama_arah))
    return None, 0, iterasi

def astar(grid, cost_grid, start, goal, numbers, total_nums):
    state_awal = State(row=start[0], col=start[1], next_num=0)
    # Priority queue isinya tuple (f_cost, state, g_cost, moves_string)
    # A* pakai f(n) = g(n) + h(n) sebagai prioritas
    pq = []
    h_awal = heuristic(state_awal, goal, numbers, total_nums)
    f_awal = 0 + h_awal    # g=0 di awal
    heapq.heappush(pq, (f_awal, state_awal, 0, ""))

    # visited: key = state, value = g_cost terkecil (sama seperti UCS)
    visited = {}
    iterasi = 0

    while pq:
        f_cost, state, g_cost, moves = heapq.heappop(pq)
        iterasi += 1

        # Cek goal
        if state.row == goal[0] and state.col == goal[1] and state.next_num ==
total_nums:
            return moves, g_cost, iterasi

        if state in visited and visited[state] <= g_cost:
            continue

```

```

        visited[state] = g_cost

        for nama_arah, arah in [("U", atas), ("D", bawah), ("L", kiri), ("R",
kanan)]:
            hasil = slide(grid, cost_grid, state, arah)
            if hasil is None:
                continue
            state_baru, move_cost = hasil
            g_baru = g_cost + move_cost
            h_baru = heuristic(state_baru, goal, numbers, total_nums)
            f_baru = g_baru + h_baru
            if state_baru not in visited or visited[state_baru] > g_baru:
                heapq.heappush(pq, (f_baru, state_baru, g_baru, moves + nama_arah))

        return None, 0, iterasi

def hitung_total_cost(grid, cost_grid, start, moves):
    """Bantu GBFS: hitung ulang total cost dari string moves."""
    arah_map = {"U": atas, "D": bawah, "L": kiri, "R": kanan}
    state = State(row=start[0], col=start[1], next_num=0)
    total = 0
    for m in moves:
        hasil = slide(grid, cost_grid, state, arah_map[m])
        if hasil is None:
            break
        state, move_cost = hasil
        total += move_cost
    return total

if __name__ == '__main__':

    # Input dari user
    nama_file = input(">> Masukkan file input: ")
    N, M, grid, cost_grid, start, goal, numbers, total_nums = read_input(nama_file)

    print(f"\nUkuran papan : {N} baris x {M} kolom")
    print(f"Start (Z)      : baris {start[0]}, kolom {start[1]}")
    print(f"Goal  (O)      : baris {goal[0]}, kolom {goal[1]}")
    print(f"Angka di peta: {numbers}")

    print("\n--- Grid ---")
    for row in grid:
        print(''.join(row))

```

```

algoritma = input("\n>> Algoritma apa yang anda pilih? (UCS/GBFS/A*): ")
algoritma = algoritma.upper() # biar "ucs" dan "UCS" sama-sama diterima

# Jalankan algoritma – timer hanya membungkus bagian ini,
# tidak termasuk baca file atau tampilkan output
if algoritma == "UCS":
    t_mulai = time.time()
    moves, total_cost, iterasi = ucs(grid, cost_grid, start, goal, total_nums)
    t_selesai = time.time()

elif algoritma == "GBFS":
    t_mulai = time.time()
    moves, total_cost, iterasi = gbfs(grid, cost_grid, start, goal, numbers,
total_nums)
    t_selesai = time.time()

elif algoritma == "A*":
    t_mulai = time.time()
    moves, total_cost, iterasi = astar(grid, cost_grid, start, goal, numbers,
total_nums)
    t_selesai = time.time()

else:
    print(f"Algoritma '{algoritma}' tidak dikenal. Pilih UCS, GBFS, atau A*.")
    exit() # berhenti di sini – moves/total_cost/iterasi belum terdefinisi

waktu_ms = (t_selesai - t_mulai) * 1000

# Tampilkan output solusi
print()
if moves is None:
    print("Tidak ada solusi!")
else:
    print(f">> Solusi      : {moves}")
    print(f">> Total cost   : {total_cost}")
    print(f">> Waktu eksekusi: {waktu_ms:.2f} ms")
    print(f">> Banyak iterasi yang dilakukan: {iterasi} iterasi")

if moves is None:
    exit()

# Playback
jawab_playback = input("\n>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
")

```

```

if jawab_playback.strip().lower() in ("ya", "y"):
    step_minta = input(f">> Pada step berapa anda ingin melakukan playback (0-
{len(moves)}): ")
    step_minta = int(step_minta)

    # Bangun list snapshot: replay ulang moves satu per satu
    arah_map = {"U": atas, "D": bawah, "L": kiri, "R": kanan}
    snapshots = []

    state_sekarang = State(row=start[0], col=start[1], next_num=0)
    snapshots.append((0, "-", state_sekarang)) # step 0 = posisi awal

    for i, m in enumerate(moves):
        hasil = slide(grid, cost_grid, state_sekarang, arah_map[m])
        state_sekarang, _ = hasil
        snapshots.append((i + 1, m, state_sekarang)) # step 1, 2, 3, ...

    # Fungsi bantu: gambar grid dengan posisi aktor ditandai @
    def gambar_grid(grid, state):
        for r in range(len(grid)):
            baris = ""
            for c in range(len(grid[0])):
                if r == state.row and c == state.col:
                    baris += "@"
                else:
                    baris += grid[r][c]
            print(baris)

    # Mulai dari step yang diminta, bisa navigasi n/p/q
    idx = step_minta
    print("\nPerintah: [n] next [p] prev [q] keluar\n")

    while True:
        step, move, state = snapshots[idx]
        print(f"\n[Step {step}/{len(moves)}] Move: {move} | Posisi:
({state.row}, {state.col}) | Next angka: {state.next_num}")
        gambar_grid(grid, state)
        cmd = input("\n[n/p/q] > ").strip().lower()
        if cmd == "n" and idx < len(snapshots) - 1:
            idx += 1
        elif cmd == "p" and idx > 0:
            idx -= 1
        elif cmd == "q":
            break

```

```
# Simpan solusi ke file
jawab_simpan = input("\n>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak): ")

if jawab_simpan.strip().lower() in ("ya", "y"):
    path_simpan = input(">> Solusi disimpan pada: ")
    f_out = open(path_simpan, "w")
    f_out.write(f"File input : {nama_file}\n")
    f_out.write(f"Algoritma : {algoritma}\n")
    f_out.write(f"Solusi      : {moves}\n")
    f_out.write(f"Total cost : {total_cost}\n")
    f_out.write(f"Waktu eksekusi: {waktu_ms:.2f} ms\n")
    f_out.write(f"Banyak iterasi yang dilakukan: {iterasi} iterasi\n")
    f_out.close()
    print(f">> Solusi disimpan pada {path_simpan}")
```

BAB IV: Pengujian dan Hasil Pengujian

1. Tabel Hasil Pengujian

<i>Test case</i>	<i>Algoritma</i>	<i>Solusi</i>	<i>Cost</i>	<i>Iterasi</i>	<i>Waktu (ms)</i>
test1.txt	UCS	RULUDRUR	87	15	0.08
test1.txt	GBFS	RULUDRUR	87	12	0.06
test1.txt	A*	RULUDRUR	87	14	0.06
test2.txt	UCS	UDRUR	70	9	0.03
test2.txt	GBFS	UDRUR	70	6	0.03
test2.txt	A*	UDRUR	70	9	0.04
test3.txt	UCS	(tidak ada)	-	5	0.02
test3.txt	GBFS	(tidak ada)	-	5	0.02
test3.txt	A*	(tidak ada)	-	5	0.02
test4.txt	UCS	DRUD	135	7	0.03
test4.txt	GBFS	DRUD	135	6	0.03
test4.txt	A*	DRUD	135	7	0.03
test5.txt	UCS	DRU	70	4	0.01
test5.txt	GBFS	RDRU	144	5	0.03
test5.txt	A*	DRU	70	4	0.02

2. Hasil Tangkapan Layar

Input File	
Input	Output

test1.txt

```
PS C:\Users\kholi\My Folders\Kuliah\semester 8\stima\Tucil3_13222071> & C:\Python314\python.exe "c:/Users/kholi/My Folders/Kuliah/semester 8/stima/Tucil3_13222071/src.py"
>> Masukan file input: test1.txt

Ukuran papan : 7 baris x 7 kolom
Start (Z)      : baris 5, kolom 1
Goal (O)       : baris 3, kolom 5
Angka di peta: {0: (1, 1), 1: (4, 1)}

--- Grid ---
XXXXXXX
X0****X
X**X**X
X***OX
X1***IX
XZ**X*X
XXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*): A*

>> Solusi      : RULUDRUR
>> Total cost  : 87
>> Waktu eksekusi: 0.27 ms
>> Banyak iterasi yang dilakukan: 14 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak): Ya
>> Pada step berapa anda ingin melakukan playback (0-8): 3

Perintah: [n] next [p] prev [q] keluar

[Step 3/8] Move: L | Posisi: (3, 1) | Next angka: 0
XXXXXXX
X0****X
X**X**X
X@***OX
X1***IX

[Step 3/8] Move: L | Posisi: (3, 1) | Next angka: 0
XXXXXXX
X0****X
X**X**X
X@***OX
X1***IX
XZ**X*X
XXXXXXX

[n/p/q] > q

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak): Ya
>> Solusi disimpan pada: solusi1.txt
>> Solusi disimpan pada solusi1.txt
PS C:\Users\kholi\My Folders\Kuliah\semester 8\stima\Tucil3_13222071>
```


test2.txt

```
ser's/kholi/My Folders/Kuliah/semester 8/stima/Tucil3_13222071/src.py
PS C:\Users\kholi\My Folders\Kuliah\semester 8\stima\Tucil3_13222071> & C:\Python314\python.exe "c:/U
sers/kholi/My Folders/Kuliah/semester 8/stima/Tucil3_13222071/src.py"
>> Masukan file input: test2.txt

Ukuran papan : 7 baris x 7 kolom
Start (Z)      : baris 5, kolom 1
Goal (O)       : baris 3, kolom 5
Angka di peta: {0: (1, 1), 1: (4, 3)}

--- Grid ---
XXXXXXX
X0****X
X**Y**X
X***OX
X**1*LX
XZ**X*X
XXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*): GBFS

>> Solusi      : UDRUR
>> Total cost  : 70
>> Waktu eksekusi: 0.10 ms
>> Banyak iterasi yang dilakukan: 6 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak): Ya
>> Pada step berapa anda ingin melakukan playback (0-5): 4

Perintah: [n] next [p] prev [q] keluar

[Step 4/5] Move: U | Posisi: (3, 3) | Next angka: 2
XXXXXXX
X0****X
X**X**X
X**1*LX
XZ**X*X
XXXXXXX

[n/p/q] > n

[Step 5/5] Move: R | Posisi: (3, 5) | Next angka: 2
XXXXXXX
X0****X
X**X**X
X***@X
X**1*LX
XZ**X*X
XXXXXXX

[n/p/q] > n

[Step 5/5] Move: R | Posisi: (3, 5) | Next angka: 2
XXXXXXX
X0****X
X**X**X
X***@X
X**1*LX
XZ**X*X
XXXXXXX

[n/p/q] > q

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak): Ya
>> Solusi disimpan pada: solusi2
>> Solusi disimpan pada solusi2
```

test3.txt	<pre> PS C:\Users\kholi\My Folders\Kuliah\semester 8\stima\Tucil3_13222071> & C:\Python314\python.exe "c:/Users/kholi/My Folders/Kuliah/semester 8/stima/Tucil3_13222071/src.py" >> Masukan file input: test3.txt Ukuran papan : 7 baris x 7 kolom Start (Z) : baris 5, kolom 1 Goal (O) : baris 5, kolom 5 Angka di peta: {0: (3, 3)} --- Grid --- XXXXXXX X*****X X*XXX*X X*XOX*X X*XXX*X XZ***OX XXXXXXX >> Algoritma apa yang anda pilih? (UCS/GBFS/A*): UCS Tidak ada solusi! >> Waktu eksekusi: 0.09 ms >> Banyak iterasi yang dilakukan: 5 iterasi </pre>
test4.txt	<pre> >> Masukan file input: test4.txt Ukuran papan : 10 baris x 10 kolom Start (Z) : baris 1, kolom 1 Goal (O) : baris 8, kolom 8 Angka di peta: {0: (1, 8)} --- Grid --- XXXXXXXXXX XZ*X***OX X*X*X*X*X X*****X*X X*X*X*X*X X*X*****X X*****X*X X*X*X*X*X X*****OX XXXXXXXXXX >> Algoritma apa yang anda pilih? (UCS/GBFS/A*): UCS >> Solusi : DRUD >> Total cost : 135 >> Waktu eksekusi: 0.09 ms >> Banyak iterasi yang dilakukan: 7 iterasi >> Apakah Anda ingin melakukan playback? (Ya/Tidak): Tidak >> Apakah Anda ingin menyimpan solusi? (Ya/Tidak): Ya >> Solusi disimpan pada: solusi4.txt >> Solusi disimpan pada solusi4.txt </pre>

test5.txt

```
PS C:\Users\kholi\My Folders\Kuliah\semester 8\stima\Tucil3_13222071> & C:\Python314\python.exe "c:/Users/kholi/My Folders/Kuliah/semester 8/stima/Tucil3_13222071/src.py"
>> Masukan file input: test5.txt

Ukuran papan : 6 baris x 8 kolom
Start (Z)      : baris 1, kolom 1
Goal (O)       : baris 1, kolom 6
Angka di peta: {}

--- Grid ---
XXXXXXXX
XZ**X*OX
X*****X
X*****X
X*****X
XXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*): GBfs

>> Solusi      : RDRU
>> Total cost  : 144
>> Waktu eksekusi: 0.10 ms
>> Banyak iterasi yang dilakukan: 5 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak): Ya
>> Pada step berapa anda ingin melakukan playback (0-4): 0

Perintah: [n] next [p] prev [q] keluar

[Step 0/4] Move: - | Posisi: (1, 1) | Next angka: 0
XXXXXXXX
X@**X*OX
X*****X
X*****X

X*****X
X*****X
XXXXXXXX

[n/p/q] > n

[Step 1/4] Move: R | Posisi: (1, 3) | Next angka: 0
XXXXXXXX
XZ* @X*OX
X*****X
X*****X
X*****X
XXXXXXXX

[n/p/q] > n

[Step 2/4] Move: D | Posisi: (4, 3) | Next angka: 0
XXXXXXXX
XZ**X*OX
X*****X
X*****X
X** @***X
XXXXXXXX

[n/p/q] > n

[Step 3/4] Move: R | Posisi: (4, 6) | Next angka: 0
XXXXXXXX
XZ**X*OX
X*****X
X*****X
X*** @X
XXXXXXXX
```

```
[n/p/q] > n

[Step 4/4] Move: U | Posisi: (1, 6) | Next angka: 0
XXXXXXXX
XZ**X*@X
X*****X
X*****X
X*****X
XXXXXXXX

[n/p/q] > q

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak): Ya
>> Solusi disimpan pada: solusi5.txt
>> Solusi disimpan pada solusi5.txt
```

3. Analisis hasil

Semua algoritma berhasil mendeteksi kasus tanpa solusi (test3.txt) dengan benar dan konsisten, ketiganya menghabiskan 5 iterasi sebelum menyatakan tidak ada solusi. Ini menunjukkan implementasi *visited-check* dan terminasi sudah benar untuk ketiga algoritma.

Pada hampir semua *test case*, GBFS konsisten melakukan iterasi paling sedikit. Pada test1.txt misalnya, GBFS hanya butuh 12 iterasi dibanding UCS yang butuh 15. Ini sesuai dengan sifat GBFS yang langsung mengejar estimasi terdekat tanpa mempertimbangkan *cost* yang sudah dikeluarkan, sehingga jarang menjelajahi *state* yang jauh dari target. A* berada di antara keduanya, yaitu lebih sedikit iterasi dari UCS, namun lebih banyak dari GBFS, karena ia mempertimbangkan $f(n) = g(n) + h(n)$ yang kadang memaksanya memeriksa beberapa alternatif sebelum mengkonfirmasi jalur terbaik.

Pada test1.txt, test2.txt, dan test4.txt, ketiga algoritma menghasilkan *cost* yang sama persis. Ini terjadi karena pada papan-papan tersebut hanya ada satu jalur solusi yang mungkin (atau semua jalur solusi memiliki *cost* yang sama), sehingga perbedaan strategi eksplorasi tidak mempengaruhi hasil akhir.

Perbedaan nyata muncul pada test5.txt: UCS dan A* menemukan solusi DRU dengan *cost* 70, sedangkan GBFS menemukan solusi RDRU dengan *cost* 144. Selisihnya sangat besar (144 vs 70, lebih dari dua kali lipat). Ini adalah bukti konkret bahwa GBFS tidak menjamin solusi optimal: ia memilih arah R terlebih dahulu karena secara heuristik (Manhattan distance) terlihat dekat ke goal, padahal jalur tersebut melewati *tile-tile* dengan *cost* tinggi (40 per *tile* di baris 1) dibanding jalur yang turun dulu ke baris bawah (*cost* 3 per *tile*).

Waktu eksekusi di semua *test case* sangat kecil (di bawah 0.1 ms) karena ukuran papan dan jumlah *state* yang dieksplorasi masih kecil. Pola umum sesuai teori: GBFS sedikit lebih cepat karena lebih sedikit iterasi, A* dan UCS sebanding tergantung struktur papan. Pada papan yang lebih besar, perbedaan waktu antara UCS dan A* akan semakin terasa karena A* dipandu heuristik dan tidak perlu mengeksplorasi semua *state* dengan *cost* rendah secara merata.

Untuk persoalan Ice Sliding Puzzle ini, A* adalah pilihan terbaik secara umum: menjamin solusi optimal seperti UCS, tetapi dengan iterasi lebih sedikit berkat panduan heuristik. GBFS cocok dipakai jika kecepatan lebih penting dari optimalitas dan struktur papan cukup sederhana. UCS menjadi pilihan terakhir karena optimal tapi paling tidak efisien dalam hal iterasi.

Lampiran

1. Checklist

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)		✓
6	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib		✓
7	Ada tambahan dua alternatif heuristik selain yang utama		✓

2. Pranala repository

https://github.com/rezkikholida/Tucil3_13222071

3. Pernyataan tidak melakukan kecurangan yang di tandatangani

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Kholida Rezki Khoiriah